pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

Logistic Regression and Odds

WEEK 2 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Week 2: real accounts, and four rules

The card file (UCI, CC BY 4.0)

30,000 accounts

Real accounts in Taiwan, payment records April to September 2005. 6,636 defaulted the following month: 22.12 percent.

Yeh, I. (2009). Default of Credit Card Clients. UCI Machine Learning Repository.

Place and period

Taiwan, 2005. Consumer credit there and then differs from US credit.

Four columns never used

SEX, EDUCATION, MARRIAGE, AGE are never model features.

Names

Model probability of default in this file. Never a credit decision for a real person.

Course 1, notebook 16 described this file with rates and counts. This week models it.

The card file and the week's split




```
train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
```

19 features

LIMIT_BAL, PAY_0 and PAY_2 to PAY_6 (codes), BILL_AMT1 to 6, PAY_AMT1 to 6 (NT dollars).

Never features

SEX, EDUCATION, MARRIAGE, AGE, and ID. An assert stops the notebook if one slips in.

Each part keeps the file's 22.12 percent default rate. The test rows are opened once, on day 10.

A straight line on a 0/1 outcome

578

validation accounts get a value below 0

12

validation accounts get a value above 1

```
line = LinearRegression().fit(X_train[["PAY_0"]], y_train)
intercept 0.2226, slope 0.1245 per PAY_0 code
validation: from -0.0264 to 1.2185
```

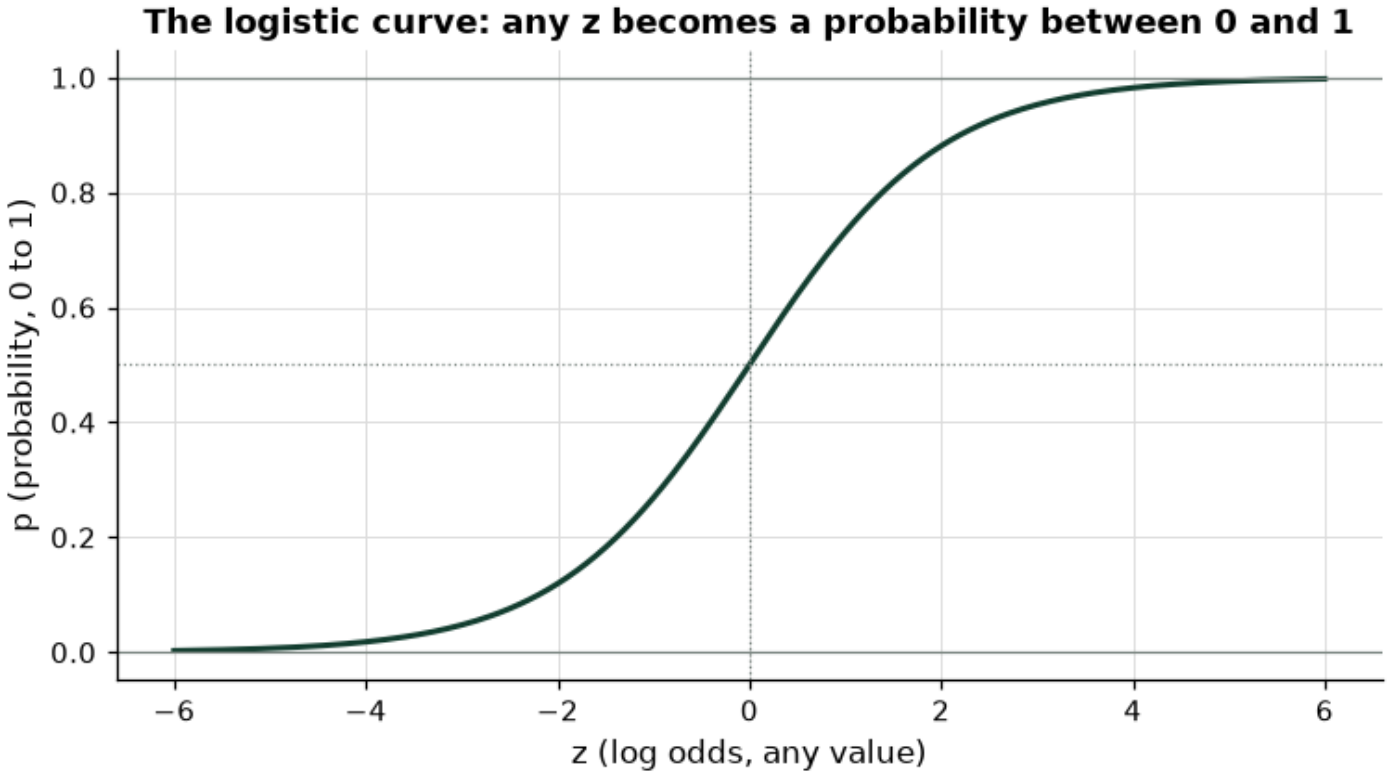
Why it fails

A straight line rises by the same amount forever. A probability must stay between 0 and 1.

The fix: a line for something that can take any value, then a curve.

It runs without a complaint. Read as probabilities, its values are impossible.

The logistic curve




```
p = 1 / (1 + exp(-z))
```

z	p
-2	0.1192
0	0.5000
2	0.8808

A logistic regression fits a straight line for z, then passes it through this curve.

Probability, odds, and log odds

Scale	Formula	Range	At 0.20
Probability	p	0 to 1	0.20
Odds	p / (1 - p)	0 to infinity	0.25
Log odds	ln(p / (1 - p))	any value	-1.3863

Code 2, training rows

odds 2.28

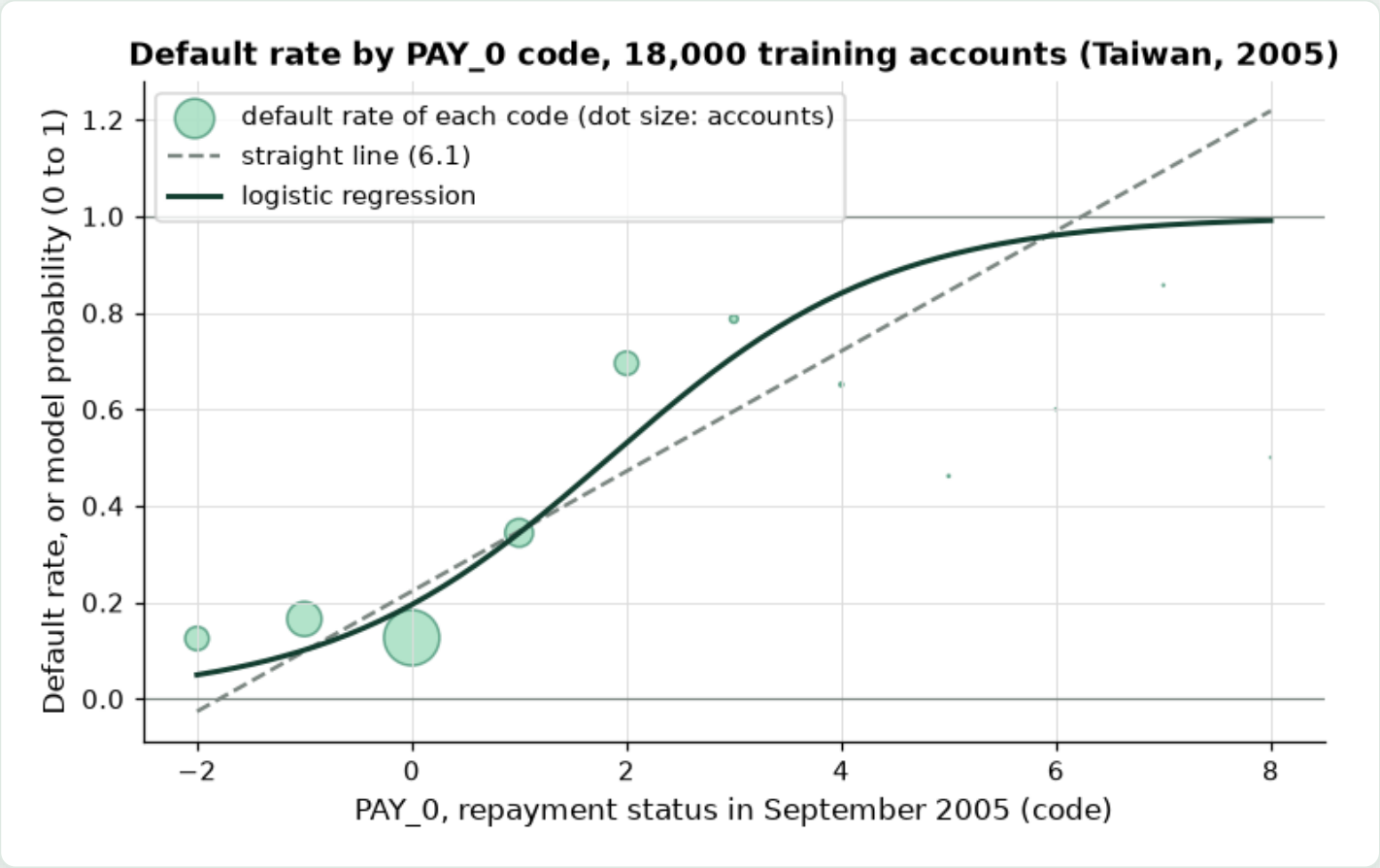
1125 defaults of 1618 accounts: rate 0.6953

PAY_0 code	Accounts	Rate	Log odds
-2	1620	0.1247	-1.9487
-1	3419	0.1653	-1.6197
0	8807	0.1264	-1.9334
1	2269	0.3438	-0.6466

The documented code -1 has the highest rate of -2, -1, 0. Codes used as one number assume even steps.

Log odds can take any value, so a line fits them. The curve is the way back to a probability.

A logistic regression with statsmodels

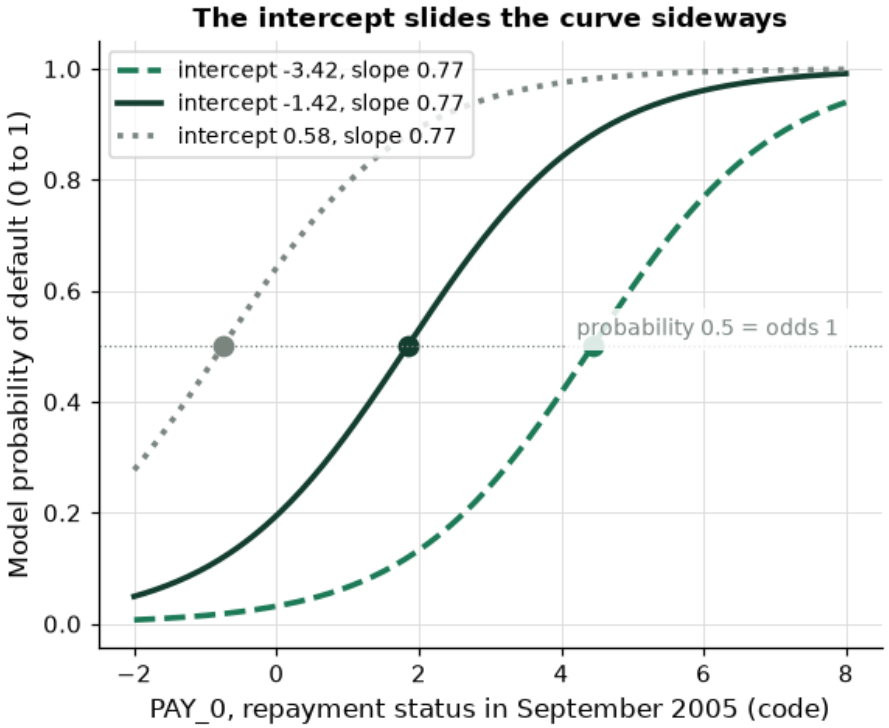


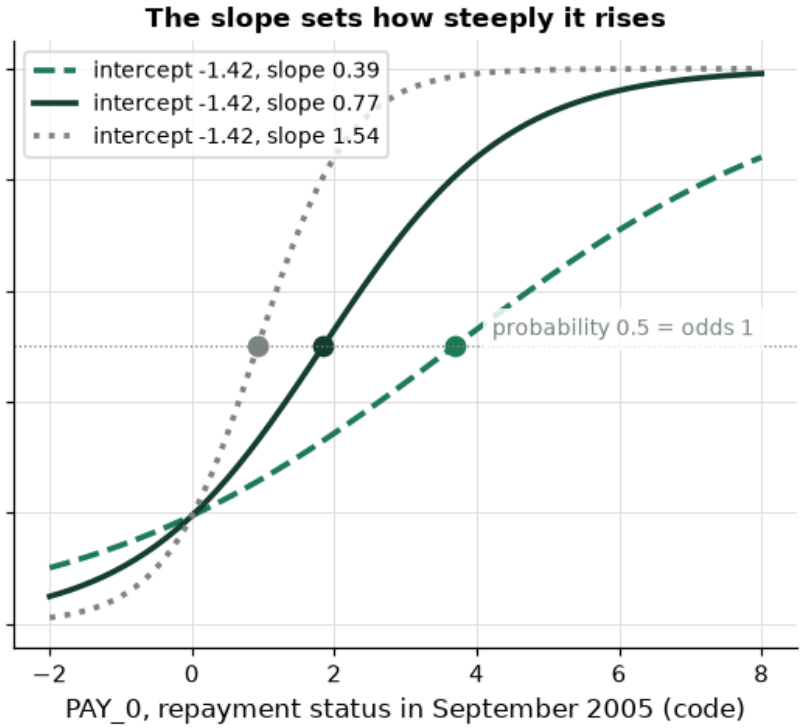

```
log odds = -1.4185 + 0.7702 x PAY_0
```

Feature	Odds ratio	95% interval
PAY_0	2.0571	1.9812 to 2.1359
LIMIT_BAL_10k	0.9799	per 10,000 NT$

Maximum likelihood on the 18,000 training rows. The curve stays between 0 and 1; neither model follows codes -2 to 0.

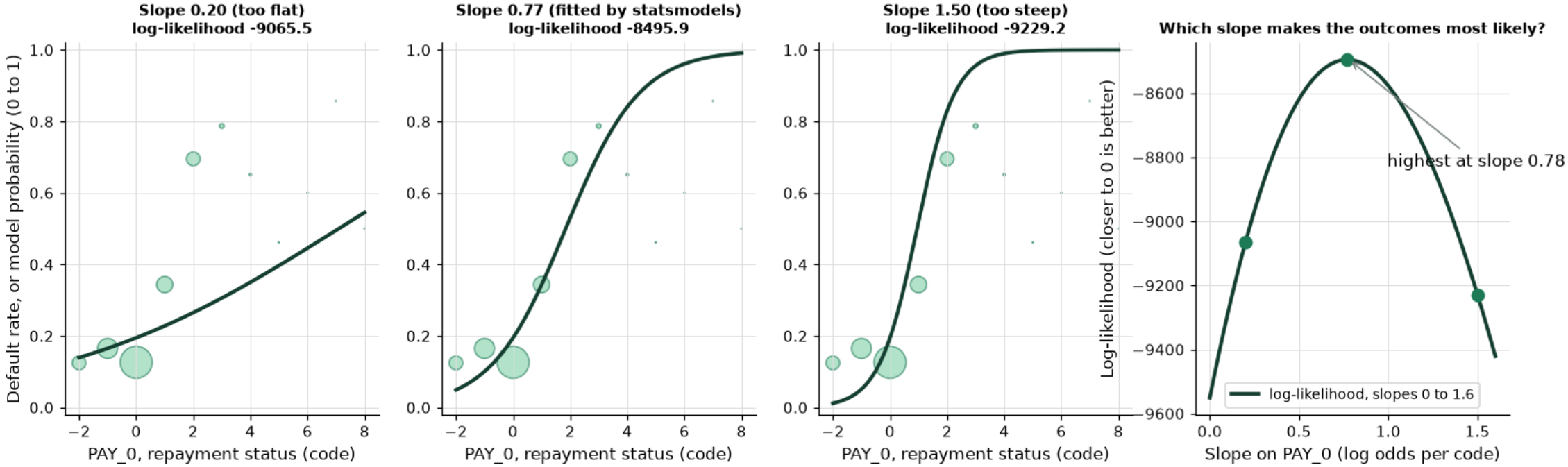
What the intercept and the slope do





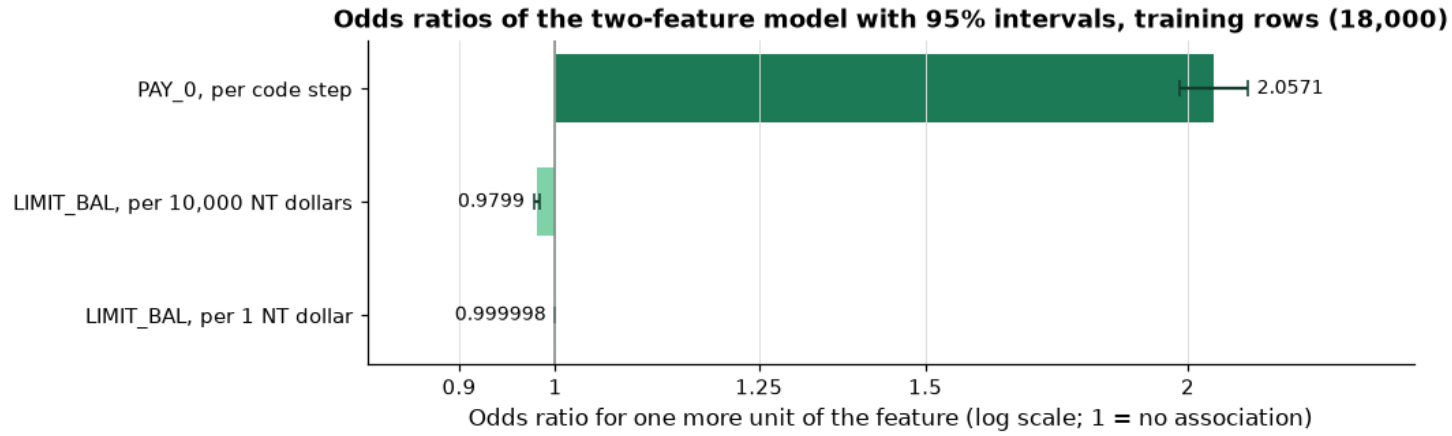
Fitted curve crosses 0.5 (odds 1) at code 1.84; intercept down 2: 4.44, up 2: -0.75. The slope sets how fast the odds change per code.

Maximum likelihood, by eye



Log-likelihood: -9065.5 too flat, -8495.9 fitted, -9229.2 too steep. Peak at 0.78; statsmodels: 0.7702.

Reading an odds ratio



PAY_0

2.0571 per step

Same limit, one code higher

LIMIT_BAL_10k

0.9799 per 10,000

About 2.0% lower odds, same code

The unit decides

0.999998 per NT$

Same coefficient: state the unit

An odds ratio is an association in this dataset, not a cause. PAY_0 alone: 2.1602.

scikit-learn: C, no penalty=, tol, and a pipeline

C

Default C=1.0 shrinks a little. C=np.inf: no penalty. PAY_0: 0.7210 against 0.7213.

No penalty=

Deprecated in scikit-learn 1.8, due to be removed in 1.10. Never used here.

tol=1e-8

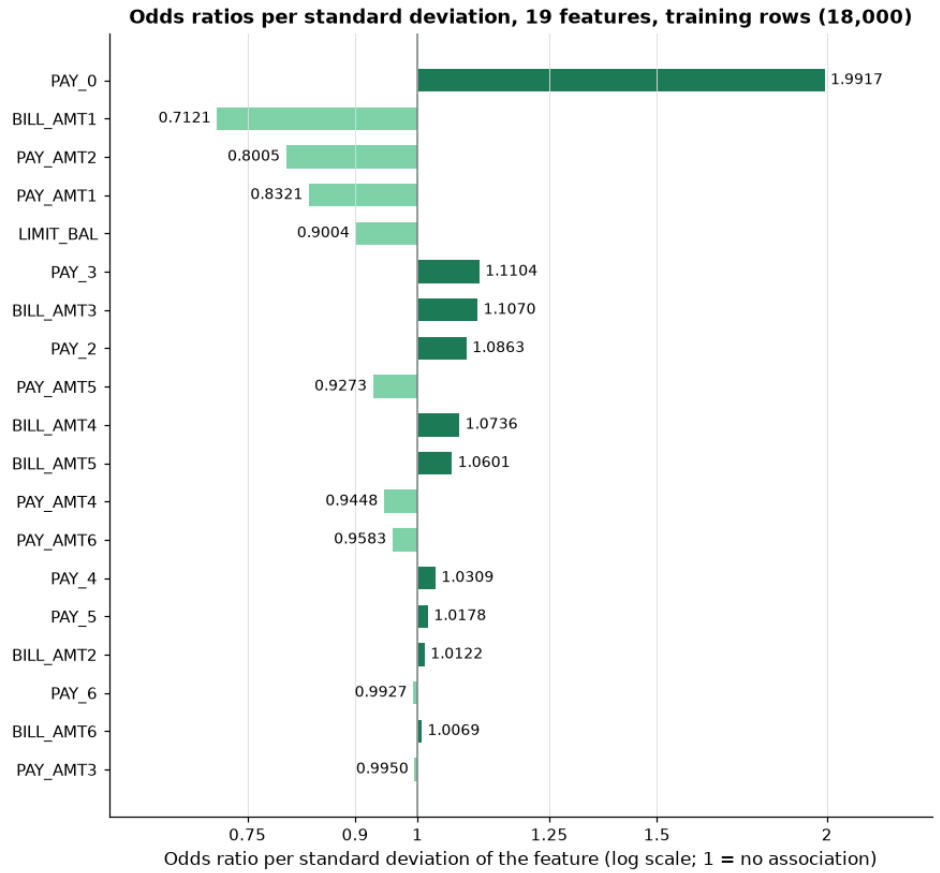
Only when compared with statsmodels (0.7213): the default can stop short.

```
Pipeline([("scale", StandardScaler()),
          ("model", LogisticRegression(C=np.inf, tol=1e-8, max_iter=1000))])
per standard deviation: 0.8052 and -0.2634
```

One SD: 1.1163 codes; 12.9563 x 10,000 NT$. Divide by it: back to statsmodels.

From today every preprocessing step lives inside a pipeline, fitted on the training rows.

All 19 columns, scaled



Per standard deviation, on one footing

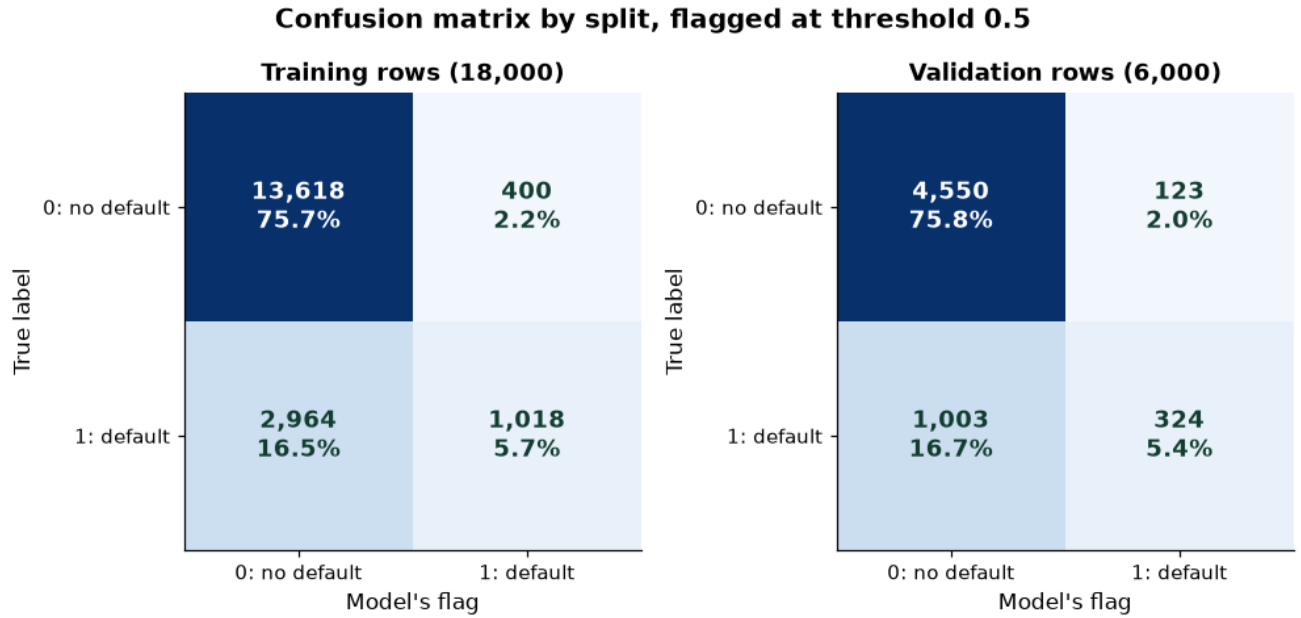
Each bar runs from 1 to the odds ratio, on a log scale. PAY_0 is furthest from 1: 1.9917 per SD.

The bills move together

BILL_AMT1 0.7121, BILL_AMT3 1.1070: opposite sides of 1. Correlation of BILL_AMT1 and BILL_AMT2: 0.9511.

Sorted by distance from 1 to read easily: not a ranking of what matters, and no bar is a cause.

A first look at flags at 0.5



Measure	Training	Validation
Accuracy	0.8131	0.8123
Recall	0.2557	0.2442
Precision	0.7179	0.7248
F1	0.3770	0.3653

Right on most accounts; the flags catch about one default in four. Day 7 builds each measure.

0.2187

mean model probability (validation)

0.2212

actual default rate (validation)

76%

of defaults below 0.5 (validation)

What the model's probability is, and is not

Allowed names

model probability of default in this file

flagged at threshold t (a first look today)

the model's score (day 8)

Never called

a credit decision, an approval, a decline

a lending policy

a risk grade for a real person

A number between 0 and 1 from a curve fitted to 18,000 accounts in Taiwan in 2005. Not about any person; not a cause; not another market.

Differences between groups in this file do not explain themselves, and they are not a basis for treating people differently.

Excel side by side: Solver

Excel side by side (tested in Excel)

`=C2/10000  (D2 down)`	limit per 10,000 NT dollars
`=1/(1+EXP(-($K$1+$K$2*B2+$K$3*D2)))  (I2 down)`	model probability
`=A2*LN(I2)+(1-A2)*LN(1-I2)  (J2 down)`	log-likelihood
`=SUM(J2:J18001)  (K4)`	the sum to maximise
`Solver: max K4 by changing K1:K3, GRG Nonlinear`	-1.1026, 0.7213, -0.0203
`=EXP(K2), =EXP(K3)`	odds ratios 2.0571, 0.9799

The box: Make Unconstrained Variables Non-Negative, ticked by default

Left ticked (PAY_0 model): intercept 0, log-likelihood -11706 against -8496, and Solver still returned 0, "found a solution".

Untick the box. Then check any optimiser's success against something you trust.

Save the day: git show --stat

1. Init

A new work folder, local settings, the guard

2. Commit

mlkit.py, test_mlkit.py, .gitignore

3. show --stat

Which files the last commit touched, and how many lines

```
git show --stat HEAD
.gitignore    |   3 +++
mlkit.py      | 242 +++
test_mlkit.py | 335 +++
3 files changed, 580 insertions(+)
```

In my_bondmath, the same command after Bring It Home shows only today's lines.

The AI check

The docstring is the prompt. The code that comes back runs, and has one modelling bug.

1

Docstring

Which rows fit, which rows report, and the units: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the coefficients come back as "odds ratios" with no exp: PAY_0 0.7213 and the limit -0.0203, below 0.

Code that runs is not the same as code that is right.

What you can do now

- A probability needs a curve: 1 / (1 + exp(-z))
- Probability, odds, and log odds: one number, three scales
- An odds ratio, with its unit, is an association in this file
- statsmodels, scikit-learn with C=np.inf, Excel with Solver
- Every preprocessing step inside a pipeline

Week 2, Day 2 The confusion matrix, precision, and recall: a probability becomes a flag